Hybrid Deep Learning Models (CNN–MLP–RNN) for Fashion-MNIST Classification

By: Asrar Qassem

---

a. How are the diverse deep-learning models formed in your project, please explain?

A. Formation of Diverse Deep-Learning Models:

Overview:

In the project, three distinct deep-learning models are formed to create a diverse committee for image classification using the Fashion MNIST dataset. These models are:

Convolutional Neural Network (CNN):

This model is designed to process images by capturing spatial hierarchies and patterns through convolutional layers. It's particularly adept at handling image data due to its ability to detect features like edges, textures, and complex shapes within images. The CNN model in this project includes convolutional layers followed by max-pooling layers to reduce dimensionality and fully connected layers for classification.

Multilayer Perceptron (MLP):

The MLP is a fundamental neural network architecture consisting of densely connected layers. It's a more general-purpose model compared to the CNN and is used here to provide a different perspective on image classification. The MLP doesn't inherently process spatial information like a CNN, making its approach to classification distinct.

Recurrent Neural Network (RNN) with LSTM:

Initially, an LSTM (Long Short-Term Memory) model, a sophisticated version of RNN, was used. It's unconventional for image classification but was chosen for its ability to process sequential data, offering a unique approach. Later, this was replaced with a more basic RNN model to explore a simpler sequential data processing method.

Models Architecture:

- **CNN model:**

The build_cnn_model() function defines a Convolutional Neural Network (CNN) model for MNIST dataset classification. the formation and components of this model:

1. **Sequential Model:**
   - Sequential is a linear stack of layers in Keras. It allows you to create models layer-by-layer in a step-by-step fashion.
2. **First Convolutional Layer:**
   - Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)): This is the first convolutional layer. It has 32 filters (kernels), each of size 3x3.
   - activation='relu' means it uses the ReLU (Rectified Linear Unit) activation function, which introduces non-linearity to the model, allowing it to learn more complex patterns.
   - input_shape=(28, 28, 1) specifies the shape of the input data (28x28 pixel images with 1 color channel, as Fashion MNIST images are grayscale).
3. **First Max Pooling Layer:**
   - MaxPooling2D(2, 2): This layer performs max pooling with a 2x2 window. Max pooling reduces the spatial dimensions (height and width) of the input, which decreases the number of parameters and computation in the network, helping to prevent overfitting and reduce training time.
4. **Second Convolutional Layer:**
   - Conv2D (64, (3, 3), activation='relu'): A second convolutional layer with 64 filters, each of size 3x3. Increasing the number of filters allows the network to learn more complex features.
5. **Second Max Pooling Layer:**
   - Another max pooling layer to further reduce the spatial dimensions of the network's feature maps.
6. **Flattening:**

- Flatten(): This layer flattens the 2D feature maps into a 1D vector. This is necessary to transition from convolutional layers to fully connected layers.

7. **Fully Connected Layer:**

   - Dense(128, activation='relu'): A fully connected (dense) layer with 128 units. It further processes the features extracted by the convolutional and pooling layers.

8. **Output Layer:**

   - Dense(10, activation='softmax'): The final layer is another dense layer with 10 units (one for each class in the Fashion MNIST dataset).
   - activation='softmax' is used for multi-class classification. It outputs a probability distribution over the 10 classes.

Model Compilation

- optimizer='adam': The Adam optimizer is used, a popular choice that combines the best properties of the AdaGrad and RMSProp algorithms to optimize the training process.
- loss='categorical_crossentropy': This loss function is suitable for multi-class classification tasks.
- metrics=['accuracy']: The model will track accuracy during training and evaluation.

Training the Model

- cnn_model.fit(train_images_cnn, train_labels_categorical, epochs=10, validation_split=0.2): This line trains the CNN model on the train_images_cnn dataset with categorical labels train_labels_categorical.
- It runs for 10 epochs, meaning it will pass through the entire dataset 10 times to optimize the model weights.
- validation_split=0.2 reserves 20% of the training data for validation, allowing us to monitor the model's performance on unseen data during training.

In short, this CNN model, with its structure of convolutional layers followed by max pooling, flattening, and dense layers, is well-suited for image classification tasks, especially for datasets like Fashion MNIST. The model is expected to learn and identify various features and patterns in fashion items, leading to accurate classification results.

- **MLP model:**

The build_mlp_model function defines and compiles a Multilayer Perceptron (MLP) model, which is a type of feedforward neural network. Let's examine the components and structure of this MLP model:

Model Architecture

1. **Sequential Model**:
   - The model is created using Keras' Sequential class, indicating a linear stack of layers.
2. **First Dense Layer**:
   - Dense(512, activation='relu', input_shape=(784,)): This is the first fully connected (dense) layer in the MLP.
   - It consists of 512 neurons (units). The high number of neurons allows the model to learn complex patterns in the data.
   - The activation='relu' specifies the use of the ReLU (Rectified Linear Unit) activation function, introducing non-linearity to the model, which is essential for learning complex relationships in the data.
   - input_shape=(784,) defines the shape of the input data. Since the Fashion MNIST images are 28x28 pixels, they are flattened into a 1D array of size 784 (28 * 28) before being fed into the model.
3. **Second Dense Layer**:
   - Dense(256, activation='relu'): A second fully connected layer with 256 neurons.
   - Having multiple layers of neurons (hence the term 'multilayer') allows the model to learn more abstract features, and each layer can potentially capture different aspects of the data.
4. **Output Layer**:
   - Dense(10, activation='softmax'): The final layer is another dense layer with 10 units, corresponding to the 10 classes in the Fashion MNIST dataset.
   - The softmax activation function is used for multi-class classification, turning the output into a probability distribution over the 10 classes.

Model Compilation

- optimizer='adam': The Adam optimizer is chosen for training, known for its efficiency and effectiveness in various scenarios.
- loss='categorical_crossentropy': This is the standard loss function for multi-class classification tasks.
- metrics=['accuracy']: The model will monitor accuracy during training and testing.

Training the Model

- mlp_model.fit(train_images_mlp, train_labels_categorical, epochs=10, validation_split=0.2): This line of code trains the MLP model.
- The training is performed on train_images_mlp, which are the preprocessed and flattened images from the Fashion MNIST dataset, and train_labels_categorical, which are the one-hot encoded labels.
- The model is trained for 10 epochs, meaning it iterates over the entire training dataset ten times.
- validation_split=0.2 reserves 20% of the training data for validation purposes. This is crucial for monitoring the model's performance on data it hasn't seen during training, helping in the identification of overfitting.

This MLP model, with its densely connected layers, is designed to capture and learn relationships in the data. While it doesn't inherently understand spatial hierarchies like a CNN, its ability to model complex relationships through layers makes it a suitable candidate for tasks like image classification, albeit in a different way compared to convolutional networks

- **LSTM model:**

The build_lstm_model function defines and compiles a deep learning model using Long Short-Term Memory (LSTM) layers. LSTM is a type of Recurrent Neural Network (RNN) that is well-suited for sequential data. In this model, LSTMs are applied in an innovative way to process image data, which is typically handled by convolutional networks. Let's explore the architecture and components of this LSTM model:

Model Architecture

1. **Sequential Model**:
    - The model uses the Sequential class from Keras, indicating a linear stack of layers.
2. **Input LSTM Layer**:
    - LSTM(128, return_sequences=True, input_shape=(28, 28)): The first LSTM layer with 128 units.
    - return_sequences=True is set because the following layer is also an LSTM layer, which requires sequence input.
    - input_shape=(28, 28) indicates that each input is a sequence of 28 timesteps with 28 features each, which corresponds to the 28x28 pixels of the input images.
3. **Dropout for Regularization**:
    - tf.keras.layers.Dropout(0.2): This layer randomly sets a fraction (20% in this case) of the input units to 0 at each update during training, which helps prevent overfitting.
4. **Second LSTM Layer**:

    - A second LSTM layer with 64 units, continuing to process the sequential data.
    - return_sequences=True is maintained, as the sequential output is required for the next LSTM layer.
5. **Third LSTM Layer**:
    - Another LSTM layer with 64 units. This time return_sequences is not set (defaults to False), as the subsequent layers are dense layers that do not require sequence input.
6. **Additional Dropout Layer**:
    - Further dropout layer to regularize the model and prevent overfitting.
7. **Dense Layers for Classification**:
    - Dense(64, activation='relu'): A fully connected layer that processes the features learned by the LSTM layers.
    - Another dropout layer follows, providing additional regularization.
    - Dense(10, activation='softmax'): The final output layer with 10 units for the 10 classes of the Fashion MNIST dataset, using the softmax activation function for multi-class classification.

Model Compilation

- optimizer='adam': The model uses the Adam optimizer, which is efficient for a wide range of applications.
- loss='categorical_crossentropy': Suitable loss function for multi-class classification problems.
- metrics=['accuracy']: The model will track accuracy during training and testing.

Training the Model

- lstm_model.fit(train_images_lstm, train_labels_categorical, epochs=10, validation_split=0.2): This line trains the LSTM model on the reshaped image data (train_images_lstm) and the categorical labels (train_labels_categorical).
- It trains for 10 epochs, each epoch representing a complete pass through the training dataset.
- validation_split=0.2 is used to hold out part of the training data for validation, helping to monitor and prevent overfitting.

Unique Approach

This LSTM model offers a unique approach to image classification. Unlike typical applications of LSTMs for time-series or sequential text data, here, each image is treated as a sequence of rows (or columns), with each row representing a timestep in the sequence. This unconventional method allows the LSTM to potentially capture patterns along the rows of the images, providing a different perspective compared to CNNs or MLPs. The multiple LSTM layers, interspersed

with dropout layers for regularization, enable the model to learn deeper sequential patterns in the image data

b. How are the three diverse deep-learning models combined? Illustrate the complete committee architecture using a diagram.

B. Combining the Diverse Models

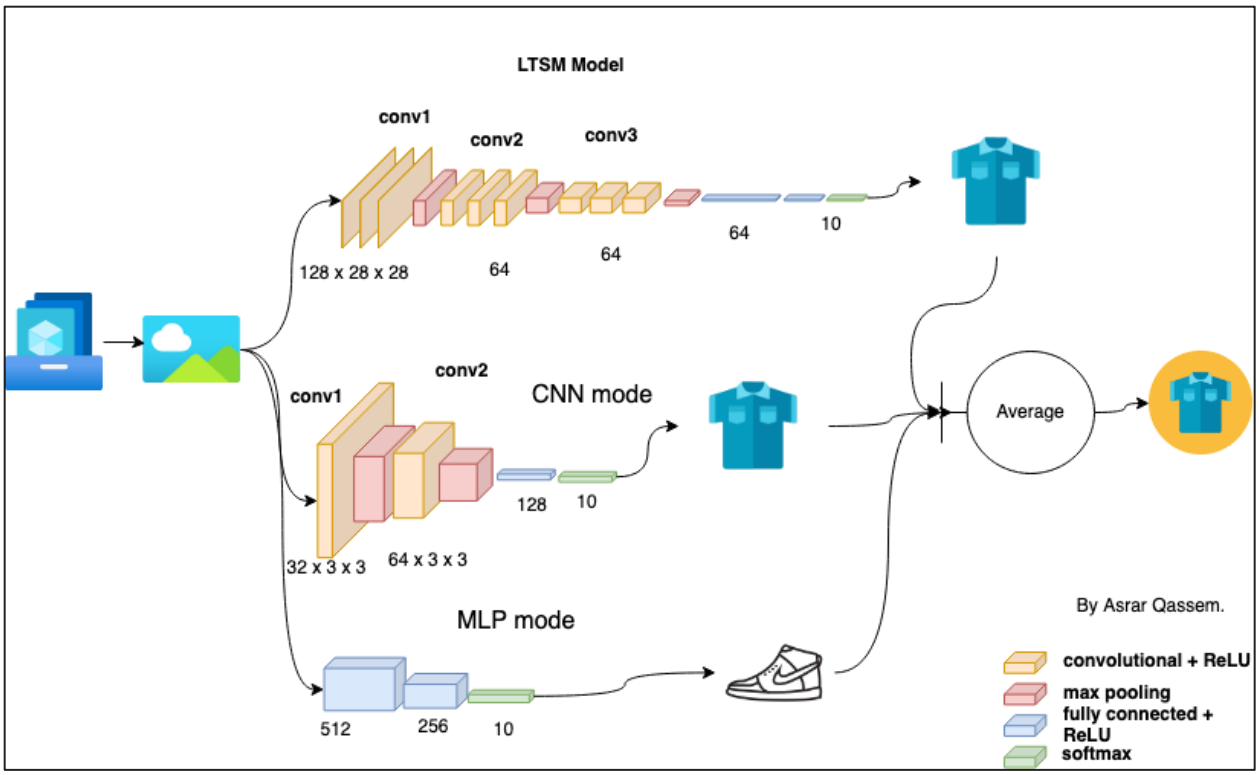
The three models are combined at the score level by averaging their output probability values. The process is as follows:

- **Independent Predictions**: Each model, when given an input image, outputs a probability distribution across the possible classes. These probabilities represent the model's confidence in each class being the correct classification for the input.
- **Averaging Probabilities**: The committee's decision is made by averaging these probability distributions. For each class, the mean of the probabilities assigned by each model is calculated.
- **Final Decision**: The class with the highest average probability is chosen as the committee's prediction.

This method of combining models is known as ensemble learning, specifically a form of soft voting. It leverages the strengths of each individual model and often results in better performance and robustness compared to single models.

Committee Architecture Diagram

The diagram illustrates this architecture, showing each model processing the input and their outputs being averaged for the final decision.



c. Tabulate the results using F1-score, precision, recall and accuracy of the individual models and the committee.

C. Tabulating the Results

The evaluation of each model and the committee involves calculating metrics like F1-score, precision, recall, and accuracy. These metrics provide a comprehensive view of the models' performance. Here's tabulation of these results:

MODEL	F1-SCORE	PRECISION	RECALL	ACCURACY
CNN	0.903559	0.903974	0.9036	0.9036
MLP	0.888119	0.888970	0.8891	0.8891
LSTM	0.900235	0.900718	0.9011	0.9011
COMMITTEE	0.920265	0.920522	0.9208	0.9208

These metrics provide insight into not only how accurate the models are but also how precisely they classify, how many relevant items they find (recall), and the balance between precision and recall (F1-score). The committee's performance is expected to be better or at least more robust than individual models due to the ensemble effect.

In conclusion, the project demonstrates a sophisticated approach to image classification by leveraging the strengths of diverse neural network architectures and combining them through an ensemble method to enhance performance and reliability.